Flight Reservation System — Basic Full-Stack Spring MVC Project

---

1. Summary of Company Requirements (from Learning.docx)

From a detailed visual inspection of every embedded image and wireframe in Learning.docx, the company assignment requires building a **Spring-based Flight Reservation System** covering 8 specific milestones:

Milestone	Feature Area	Mandatory Company Requirements & Wireframe Details
Milestone 1	**Home & Search Flights**	Landing page with a **Search Flights** card (From, To) and a **User Actions** card (Login, Sign Up). On searching, displays an **Available Flights** table showing Flight Number, Route, Schedule (Dep/Arr), Price, and an actionable **[Book]** button. If guest clicks Book, redirects to Login; after login, redirects directly to Booking Form.
Milestone 2	**User Management**	**Sign Up Page**: Email, Password (length > 8), Role dropdown (User, Admin). Validations displayed. **Login Page**: Email & Password validation. Redirects to Admin Dashboard or User Dashboard according to role.
Milestone 3	**Book Flight Workflow**	**1. Booking Form**: Shows flight number banner, select Journey Date (HTML5 date picker), and Number of Passengers (1–10). Verifies available seats (capacity - bookedSeats >= requested). If insufficient, displays error alert; else proceeds to Step 2. **2. Add Passenger Screens**: Shown sequentially for each passenger (e.g. *Add Passenger 1*, *Add Passenger 2*), showing remaining count and progress bar. Last passenger shows **[Complete Booking]** button. **3. Booking Confirmation**: Displays congratulations, generated Booking ID (from tbl_bookings), next-steps checklist, and buttons for [Go to Dashboard] and [Print Confirmation].
Milestone 4	**Admin Dashboard**	**Dashboard**: Quick links for **[Add Flight]** and **[View Flights]**. **Add Flight**: Flight Number (unique), Capacity (default 20), Source, Destination, Departure Time, Arrival Time, Price per Seat. **View Flights**: Displays **Flight Journey Status** with two tabs: *Upcoming Journeys* and *Completed Journeys*. Shows Flight Number, Journey Date, Total Passengers, Total Bookings, Total Revenue, and Days Difference. Only flights with bookings are displayed.
Milestone 5	**User Dashboard**	**Dashboard**: Cards for **[My Bookings]** and **[Search Flights]**. **My Bookings**: Lists user's bookings with Booking ID, Flight Number, Journey Date, Status (BOOKED / CANCELLED), [View Tickets] button, and [Cancel] button. Cancelling releases seats and disables the cancel button. **View Tickets**: Displays individual airline ticket cards with PNR, Passenger Name, Age, Gender, Journey Date, assigned Seat Number (e.g., 12A, 12B), Booking ID, and Status.

Milestone	Feature Area	Mandatory Company Requirements & Wireframe Details
Milestone 6	**Global Exception Handling**	Implemented using @ControllerAdvice in the controller package, rendering a clean, user-friendly error.jsp.
Milestone 7	**Unit Testing**	Unit tests for Sign-Up and Login workflows verifying success, duplicate email, short password, and invalid credentials.
Milestone 8	**Version Control**	Complete setup and instructions to push the code to a Git repository.

---

2. Why This Matches Your Training

Your trainer demonstrated a Customer Banking application (Spring-Web-Demo-New) to teach you:

1. **Spring MVC Architecture**: Controller →\rightarrow→ Service →\rightarrow→ DAO →\rightarrow→ Database.
2. **Java-based Spring Configuration**: @Configuration, @ComponentScan, @Bean for ViewResolver, DataSource (DriverManagerDataSource), and JdbcTemplate.
3. **Dispatching**: WebAppInitializer extending AbstractAnnotationConfigDispatcherServletInitializer.
4. **Data Access**: Spring JdbcTemplate with inline lambda RowMapper implementations and queryForObject/query/update.
5. **Presentation**: Standard JSP views using JSTL (c:forEach, c:if, c:choose) served by InternalResourceViewResolver (/WEB-INF/views/*.jsp).
6. **Session Management**: HttpSession for authentication and multi-step workflows.

The **Flight Reservation System** has been built using the **exact same patterns, syntax, and annotations** you learned in class, without introducing Spring Boot, Spring Security, Hibernate, or complex microservice layers.

---

3. Package & Directory Structure

The project strictly follows the mandatory six packages under com.deloitte:

text

Spring-Web-Demo-New/

├── pom.xml # Maven build file (Tomcat 9 compatible)

├── flightdb.sql # Database schema & sample records

├── src/

| ├── main/

```
|   |   ├── java/
|   |   |   └── com.deloitte/
|   |   |       ├── config/
|   |   |       |   ├── AppConfig.java              # Spring MVC, DataSource, JdbcTemplate
|   |   |       |   └── WebAppInitializer.java      # DispatcherServlet registration
|   |   |       ├── controller/
|   |   |       |   ├── AdminController.java        # Add flight & view flight journey status
|   |   |       |   ├── BookingController.java      # Booking form, sequential passengers, tickets
|   |   |       |   ├── GlobalExceptionHandler.java  # @ControllerAdvice centralized errors
|   |   |       |   ├── HomeController.java         # Landing page & flight search
|   |   |       |   └── UserController.java         # Sign up, login, logout, profile
|   |   |       ├── dao/
|   |   |       |   ├── BookingDao.java
|   |   |       |   ├── BookingDaoImpl.java         # JdbcTemplate + KeyHolder + RowMapper
|   |   |       |   ├── FlightDao.java
|   |   |       |   ├── FlightDaoImpl.java          # Flight search & CRUD queries
|   |   |       |   ├── TicketDao.java
|   |   |       |   ├── TicketDaoImpl.java          # Ticket generation & seat counting
|   |   |       |   ├── UserDao.java
|   |   |       |   └── UserDaoImpl.java            # User retrieval & registration queries
|   |   |       ├── dto/
|   |   |       |   ├── BookingDraftDto.java        # Holds multi-step booking data in session
|   |   |       |   ├── FlightSearchDto.java        # Flight search query object
|   |   |       |   ├── FlightSummaryDto.java       # Aggregated journey statistics for admin
|   |   |       |   ├── LoginDto.java               # User login form data
|   |   |       |   ├── PassengerDto.java           # Passenger details form data
|   |   |       |   └── SignUpDto.java              # User sign-up form data
|   |   |       ├── model/
|   |   |       |   ├── Booking.java                # Maps to tbl_bookings
|   |   |       |   ├── Flight.java                 # Maps to tbl_flights
```

```
|   |   |   |   ├── Ticket.java                # Maps to tbl_tickets
|   |   |   |   └── User.java                  # Maps to tbl_users
|   |   |   └── service/
|   |   |       ├── BookingService.java
|   |   |       ├── BookingServiceImpl.java        # Seat calculations, bookings & releases
|   |   |       ├── FlightService.java
|   |   |       ├── FlightServiceImpl.java         # Flight management & search logic
|   |   |       ├── UserService.java
|   |   |       └── UserServiceImpl.java           # Authentication & validation logic
|   |   └── webapp/
|   |       └── WEB-INF/
|   |           ├── web.xml                       # Servlet 4.0 descriptor for Tomcat 9
|   |           └── views/
|   |               ├── add-flight.jsp             # Admin Add Flight page
|   |               ├── add-passenger.jsp          # Sequential Add Passenger form
|   |               ├── admin-dashboard.jsp        # Admin portal
|   |               ├── booking-confirmation.jsp   # Booking confirmed screen
|   |               ├── booking-form.jsp           # Date and passenger count form
|   |               ├── error.jsp                  # Friendly error display
|   |               ├── home.jsp                   # Landing page + Search results
|   |               ├── login.jsp                  # Login page
|   |               ├── my-bookings.jsp            # User bookings & cancellation
|   |               ├── profile.jsp                # User profile & password update
|   |               ├── signup.jsp                 # Sign up page
|   |               ├── user-dashboard.jsp         # User portal
|   |               ├── view-flights.jsp           # Upcoming & Completed flights tabs
|   |               └── view-tickets.jsp           # Airline ticket cards with seat & PNR
|   └── test/
|       └── java/
|           └── com.deloitte/service/
```

| └── UserServiceTest.java # JUnit 4 tests for Sign-Up & Login

---

4. Key Source Files

Configuration Files

pom.xml

Configured with Java 25 compiler, Spring 5.3.39, MySQL Connector 8.0.33, JSTL 1.2, javax.servlet-api (provided for Tomcat 9), and JUnit 4.13.2.

xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>
        <groupId>Spring-Web-Demo-New</groupId>
        <artifactId>Spring-Web-Demo-New</artifactId>
        <version>0.0.1-SNAPSHOT</version>
        <packaging>war</packaging>
        <dependencies>
                <dependency>
                        <groupId>org.springframework</groupId>
                        <artifactId>spring-jdbc</artifactId>
                        <version>5.3.39</version>
                        <scope>compile</scope>
                </dependency>
                <dependency>
                        <groupId>mysql</groupId>
                        <artifactId>mysql-connector-java</artifactId>
                        <version>8.0.33</version>
                </dependency>
                <dependency>
                        <groupId>org.springframework</groupId>
                        <artifactId>spring-context</artifactId>
```

```
            <version>5.3.39</version>
            <scope>compile</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-webmvc</artifactId>
            <version>5.3.39</version>
            <scope>compile</scope>
        </dependency>
        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>jstl</artifactId>
            <version>1.2</version>
            <scope>compile</scope>
        </dependency>
        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>javax.servlet-api</artifactId>
            <version>4.0.1</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
```

```
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.13.0</version>
                <configuration>
                    <release>25</release>
                </configuration>
            </plugin>
            <plugin>
                <artifactId>maven-war-plugin</artifactId>
                <version>3.2.3</version>
                <configuration>
                    <failOnMissingWebXml>false</failOnMissingWebXml>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

AppConfig.java

java

```
package com.deloitte.config;
import javax.sql.DataSource;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.web.servlet.ViewResolver;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.view.InternalResourceViewResolver;
@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "com.deloitte")
```

```
public class AppConfig {
	@Bean
	public ViewResolver viewResolver() {
		InternalResourceViewResolver viewResolver = new InternalResourceViewResolver();
		viewResolver.setPrefix("/WEB-INF/views/");
		viewResolver.setSuffix(".jsp");
		return viewResolver;
	}
	@Bean
	public DataSource dataSource() {
		DriverManagerDataSource dataSource = new DriverManagerDataSource();
		dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");

	dataSource.setUrl("jdbc:mysql://localhost:3306/flightdb?useSSL=false&allowPublicKeyRetrie
val=true&serverTimezone=UTC");
		dataSource.setUsername("root");
		dataSource.setPassword("Thomas@sql");
		return dataSource;
	}
	@Bean
	public JdbcTemplate jdbcTemplate() {
		return new JdbcTemplate(dataSource());
	}
}
```

---

5. Database Setup Script

A dedicated SQL script file flightdb.sql is created in the project root:

```
sql
DROP DATABASE IF EXISTS flightdb;
CREATE DATABASE flightdb;
USE flightdb;
CREATE TABLE tbl_users
```

```
(
    user_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL CHECK(LENGTH(password) > 8),
    role VARCHAR(50) NOT NULL CHECK(role IN ('user', 'admin'))
) AUTO_INCREMENT = 1001;
CREATE TABLE tbl_flights
(
    flight_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    flight_number VARCHAR(50) NOT NULL UNIQUE,
    source VARCHAR(50) NOT NULL,
    destination VARCHAR(50) NOT NULL,
    departure_time VARCHAR(50) NOT NULL,
    arrival_time VARCHAR(50) NOT NULL,
    capacity INT CHECK(capacity > 0),
    price_per_seat DOUBLE CHECK(price_per_seat > 0)
) AUTO_INCREMENT = 5001;
CREATE TABLE tbl_bookings
(
    booking_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    booking_date DATE NOT NULL,
    journey_date DATE NOT NULL,
    seats INT CHECK(seats > 0),
    total_price DOUBLE CHECK(total_price > 0),
    status VARCHAR(50) CHECK(status IN ('confirmed', 'cancelled')),
    flight_id BIGINT,
    user_id BIGINT
) AUTO_INCREMENT = 90001;
CREATE TABLE tbl_tickets
(
    pnr_no BIGINT PRIMARY KEY AUTO_INCREMENT,
```

```
    passenger_name VARCHAR(50) NOT NULL,
    gender VARCHAR(50) CHECK(gender IN ('male', 'female', 'other')),
    age INT CHECK(age BETWEEN 0 AND 120),
    journey_date DATE NOT NULL,
    seat_no VARCHAR(20) NOT NULL,
    booking_id BIGINT,
    status VARCHAR(50) CHECK(status IN ('confirmed', 'cancelled'))
) AUTO_INCREMENT = 100001;
ALTER TABLE tbl_bookings
ADD CONSTRAINT fk_booking_flight FOREIGN KEY(flight_id) REFERENCES tbl_flights(flight_id),
ADD CONSTRAINT fk_booking_user FOREIGN KEY(user_id) REFERENCES tbl_users(user_id),
ADD CONSTRAINT chk_journey_date CHECK(journey_date >= booking_date);
ALTER TABLE tbl_tickets
ADD CONSTRAINT fk_ticket_booking FOREIGN KEY(booking_id) REFERENCES
tbl_bookings(booking_id);
-- Default Admin and Sample Users
INSERT INTO tbl_users (email, password, role) VALUES
('admin@flightapp.com', 'Admin@2026', 'admin'),
('ravi.kumar@example.com', 'Ravi@2026', 'user'),
('meera.nair@example.com', 'Meera@2026', 'user');
-- Default Flights
INSERT INTO tbl_flights (flight_number, source, destination, departure_time, arrival_time, capacity,
price_per_seat) VALUES
('AI101', 'Delhi', 'Mumbai', '06:00', '08:10', 180, 4500),
('6E202', 'Mumbai', 'Bangalore', '09:30', '11:15', 150, 3800),
('SG303', 'Bangalore', 'Chennai', '14:00', '15:00', 120, 2500);
```

---

6. Setup & Execution Instructions in Eclipse + Tomcat 9.0

Step 1: Execute SQL in MySQL

1. Open **MySQL Workbench** or your MySQL command-line client.
2. Open the file flightdb.sql located inside the project folder.
3. Execute the script to create the database, tables, constraints, and default records.

4. Verify your MySQL password in AppConfig.java (currently configured as username: root, password: Thomas@sql).

Step 2: Import into Eclipse

1. Launch **Eclipse IDE for Enterprise Java and Web Developers**.
2. Go to **File →\rightarrow→ Import...**
3. Select **Maven →\rightarrow→ Existing Maven Projects** and click **Next**.
4. In **Root Directory**, browse and select: c:\Users\DELL\Downloads\Practice_java-main\Practice_java-main\Spring-Web-Demo-New (2)\Spring-Web-Demo-New
5. Ensure pom.xml is checked and click **Finish**.
6. Wait for Eclipse to finish building and downloading dependencies.
7. Right-click the project in Project Explorer →\rightarrow→ **Maven →\rightarrow→ Update Project...** →\rightarrow→ Click **OK**.

Step 3: Configure Apache Tomcat 9.0

1. In the **Servers** tab at the bottom of Eclipse (if not visible, go to **Window →\rightarrow→ Show View →\rightarrow→ Servers**).
2. Right-click in the Servers view →\rightarrow→ **New →\rightarrow→ Server**.
3. Select **Apache →\rightarrow→ Tomcat v9.0 Server** →\rightarrow→ click **Next**.
4. Browse to your Tomcat 9.0 installation directory (if prompted) and select JDK 25 / your installed JRE.
5. In the **Add and Remove** dialog, select **Spring-Web-Demo-New** from the left pane, click **Add >**, and click **Finish**.

Step 4: Run the Application

1. In the **Servers** tab, right-click **Tomcat v9.0 Server** and select **Start**.
2. Open your web browser and navigate to:

text

http://localhost:8080/Spring-Web-Demo-New/home

3. Test accounts ready to use:
    - **Admin Account**: admin@flightapp.com / Admin@2026
    - **User Account**: ravi.kumar@example.com / Ravi@2026

---

7. How the Frontend, Backend, and Database Interact

---

8. Java Concepts Mapped to Your Training

Java & Spring Concept	Where Used in Project	Training Reference File
DispatcherServlet & Web Initializer	WebAppInitializer.java	WebAppInitializer.java in original starter
@Configuration, @Bean, @ComponentScan	AppConfig.java	AppConfig.java in original starter
Dependency Injection & @Autowired	Used in Controllers, Services, and DAOs for constructor and field injection	Demonstrated throughout CustomerController.java & CustomerDaoImpl.java
JdbcTemplate & inline RowMapper	UserDaoImpl.java, FlightDaoImpl.java, BookingDaoImpl.java	CustomerDaoImpl.java lambda (rs, rowNum) -> ...
Spring MVC Form & Session Handling	BookingController.java, UserController.java	CustomerController.java (HttpSession session)
Centralized @ControllerAdvice	GlobalExceptionHandler.java	Milestone 6 Best Practices Requirement
Unit Testing (JUnit 4)	UserServiceTest.java	Milestone 7 Unit Testing Requirement

---

9. Company Requirements Completion Checklist

- **Milestone 1**: Landing page (home.jsp) with Search Flights and User Actions cards.
- **Milestone 1**: List available flights with Flight, Route, Schedule, Price, and Book action.
- **Milestone 1**: Clicking Book while logged out redirects to login, then resumes booking.
- **Milestone 2**: Sign Up with unique email, password (> 8 chars), and role (User/Admin).
- **Milestone 2**: Login with credential validation and role-based redirect.
- **Milestone 3**: Booking Form with journey date and passenger count (1–10).
- **Milestone 3**: Seat availability calculation (capacity - booked >= requested).
- **Milestone 3**: Sequential Add Passenger forms with remaining count and progress bar.
- **Milestone 3**: Booking Confirmation screen with Booking ID, badges, and next steps.
- **Milestone 4**: Admin Dashboard with Add Flight and View Flights options.
- **Milestone 4**: Add Flight page with default capacity 20 and unique flight number validation.
- **Milestone 4**: View Flights page with Upcoming and Completed tabs showing passengers, bookings, revenue, and days difference.

- **Milestone 5**: User Dashboard with My Bookings and Search Flights options.
- **Milestone 5**: My Bookings page showing booking status and View Tickets action.
- **Milestone 5**: Cancel Ticket function that releases seats and disables the cancel button.
- **Milestone 5**: View Tickets page generating airline ticket cards with PNR, seat numbers, and print option.
- **Milestone 6**: Global exception handling with @ControllerAdvice and error.jsp.
- **Milestone 7**: Unit testing for Sign-Up and Login screens in UserServiceTest.java.
- **Tomcat 9 Compatibility**: Uses javax.servlet API, Servlet 4.0 descriptor, and standard WAR structure.
- **Package Strictness**: Exactly the six allowed packages under com.deloitte.

---

10. Assessment Viva Preparation Guide

Q1: What is the flow of a request in this Spring MVC application?

Answer: A request hits Tomcat and is routed to the DispatcherServlet (configured in WebAppInitializer). The DispatcherServlet consults handler mappings to invoke the appropriate @Controller method. The controller calls the @Service layer for business logic (like seat availability or credential checks), which uses the @Repository DAO layer with JdbcTemplate to query MySQL. The controller returns a logical view name (e.g. "home"), which InternalResourceViewResolver resolves to /WEB-INF/views/home.jsp with model attributes.

Q2: How do you ensure seat numbers and seat availability remain accurate?

Answer: In BookingServiceImpl, available seats are dynamically computed as flight.getCapacity() - bookedSeatsCount. The bookedSeatsCount only sums bookings where status = 'confirmed' for that specific flight and journey date. When a booking is cancelled, its status is updated to 'cancelled', which immediately releases those seats back into the available pool for other users.

Q3: Why did you use javax.servlet instead of jakarta.servlet?

Answer: Apache Tomcat 9.0 implements Java EE 8 (Servlet 4.0 specification), which uses the javax.servlet namespace. The jakarta.servlet namespace was introduced in Jakarta EE 9 (Tomcat 10+). Using javax.servlet ensures 100% native compatibility with Tomcat 9 without ClassNotFound errors.

Q4: How does the sequential Add Passenger screen work?

Answer: When the booking form is submitted, a BookingDraftDto is stored in the HttpSession with the selected flight, journey date, and target passenger count. On each GET request to /booking/passenger, the controller checks how many passengers have already been added to the session list. If fewer than requested, it displays *Passenger X of Y* with an [Add Passenger & Continue] button. When the final passenger is reached, the button changes to [Complete Booking], which executes the database transaction, inserts the booking and tickets, clears the session draft, and redirects to the confirmation page.

---

11. Environment Verification Summary

- **Code Validation**: All 32 Java source files, 14 JSP templates, XML descriptors, and the SQL script have been verified for syntax correctness, matching braces, valid imports, and matching method signatures.
- **Local Database Step**: Because the MySQL service was not actively listening on port 3306 in this CLI session, please ensure you execute flightdb.sql in MySQL Workbench and start your local MySQL server prior to starting Tomcat in Eclipse.

```
text

Spring-Web-Demo-New/
├── pom.xml                                    # Maven build file (Tomcat 9 compatible)
├── flightdb.sql                               # Database schema & sample records
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com.deloitte/
│   │   │       ├── config/
│   │   │       │   ├── AppConfig.java             # Spring MVC, DataSource, JdbcTemplate
│   │   │       │   └── WebAppInitializer.java     # DispatcherServlet registration
│   │   │       ├── controller/
│   │   │       │   ├── AdminController.java       # Add flight & view flight journey status
│   │   │       │   ├── BookingController.java     # Booking form, sequential passengers, tickets
│   │   │       │   ├── GlobalExceptionHandler.java # @ControllerAdvice centralized errors
│   │   │       │   ├── HomeController.java        # Landing page & flight search
│   │   │       │   └── UserController.java        # Sign up, login, logout, profile
```

```
│   │   │       │   └── UserController.java        # Sign up, login, logout, profile
│   │   │       ├── dao/
│   │   │       │   ├── BookingDao.java
│   │   │       │   ├── BookingDaoImpl.java        # JdbcTemplate + KeyHolder + RowMapper
│   │   │       │   ├── FlightDao.java
│   │   │       │   ├── FlightDaoImpl.java         # Flight search & CRUD queries
│   │   │       │   ├── TicketDao.java
│   │   │       │   ├── TicketDaoImpl.java         # Ticket generation & seat counting
│   │   │       │   ├── UserDao.java
│   │   │       │   └── UserDaoImpl.java           # User retrieval & registration queries
│   │   │       ├── dto/
│   │   │       │   ├── BookingDraftDto.java       # Holds multi-step booking data in session
│   │   │       │   ├── FlightSearchDto.java       # Flight search query object
│   │   │       │   ├── FlightSummaryDto.java      # Aggregated journey statistics for admin
│   │   │       │   ├── LoginDto.java              # User login form data
│   │   │       │   ├── PassengerDto.java          # Passenger details form data
│   │   │       │   └── SignUpDto.java             # User sign-up form data
│   │   │       ├── model/
│   │   │       │   ├── Booking.java               # Maps to tbl_bookings
│   │   │       │   ├── Flight.java                # Maps to tbl_flights
│   │   │       │   ├── Ticket.java                # Maps to tbl_tickets
│   │   │       │   └── User.java                  # Maps to tbl_users
```

```
        ├── dto/
        │   ├── BookingDraftDto.java          # Holds multi-step booking data in session
        │   ├── FlightSearchDto.java          # Flight search query object
        │   ├── FlightSummaryDto.java         # Aggregated journey statistics for admin
        │   ├── LoginDto.java                 # User login form data
        │   ├── PassengerDto.java             # Passenger details form data
        │   └── SignUpDto.java                # User sign-up form data
        ├── model/
        │   ├── Booking.java                  # Maps to tbl_bookings
        │   ├── Flight.java                   # Maps to tbl_flights
        │   ├── Ticket.java                   # Maps to tbl_tickets
        │   └── User.java                     # Maps to tbl_users
        └── service/
            ├── BookingService.java
            ├── BookingServiceImpl.java       # Seat calculations, bookings & releases
            ├── FlightService.java
            ├── FlightServiceImpl.java        # Flight management & search logic
            ├── UserService.java
            └── UserServiceImpl.java          # Authentication & validation logic
    └── webapp/
```

```
            └── UserServiceImpl.java          # Authentication & validation logic
    └── webapp/
        └── WEB-INF/
            ├── web.xml                       # Servlet 4.0 descriptor for Tomcat 9
            └── views/
                ├── add-flight.jsp            # Admin Add Flight page
                ├── add-passenger.jsp         # Sequential Add Passenger form
                ├── admin-dashboard.jsp       # Admin portal
                ├── booking-confirmation.jsp  # Booking confirmed screen
                ├── booking-form.jsp          # Date and passenger count form
                ├── error.jsp                 # Friendly error display
                ├── home.jsp                  # Landing page + Search results
                ├── login.jsp                 # Login page
                ├── my-bookings.jsp           # User bookings & cancellation
                ├── profile.jsp               # User profile & password update
                ├── signup.jsp                # Sign up page
                ├── user-dashboard.jsp        # User portal
                ├── view-flights.jsp          # Upcoming & Completed flights tabs
                └── view-tickets.jsp          # Airline ticket cards with seat & PNR
    └── test/
        └── java/
            └── com.deloitte/service/
                └── UserServiceTest.java      # JUnit 4 tests for Sign-Up & Login
```

```
            ├── view-flights.jsp              # Upcoming & Completed flights tabs
            └── view-tickets.jsp              # Airline ticket cards with seat & PNR
    └── test/
        └── java/
            └── com.deloitte/service/
                └── UserServiceTest.java      # JUnit 4 tests for Sign-Up & Login
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
	xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
	<modelVersion>4.0.0</modelVersion>
	<groupId>Spring-Web-Demo-New</groupId>
	<artifactId>Spring-Web-Demo-New</artifactId>
	<version>0.0.1-SNAPSHOT</version>
	<packaging>war</packaging>
	<dependencies>
		<dependency>
			<groupId>org.springframework</groupId>
			<artifactId>spring-jdbc</artifactId>
			<version>5.3.39</version>
			<scope>compile</scope>
		</dependency>
		<dependency>
			<groupId>mysql</groupId>
			<artifactId>mysql-connector-java</artifactId>
			<version>8.0.33</version>
		</dependency>
		<dependency>
			<groupId>org.springframework</groupId>
			<artifactId>spring-context</artifactId>
			<version>5.3.39</version>
			<scope>compile</scope>
		</dependency>
		<dependency>
			<groupId>org.springframework</groupId>
			<artifactId>spring-webmvc</artifactId>
```

```
            <version>5.3.39</version>
            <scope>compile</scope>
        </dependency>
        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>jstl</artifactId>
            <version>1.2</version>
            <scope>compile</scope>
        </dependency>
        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>javax.servlet-api</artifactId>
            <version>4.0.1</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.13.0</version>
                <configuration>
                    <release>25</release>
                </configuration>
            </plugin>
```

```
                <plugin>
                    <artifactId>maven-war-plugin</artifactId>
                    <version>3.2.3</version>
                    <configuration>
                        <failOnMissingWebXml>false</failOnMissingWebXml>
                    </configuration>
                </plugin>
            </plugins>
        </build>
</project>
```



